

JAVA 17 OCP · YDS STUDY SERIES

Unit 02 · Technical Memory Notes · Operators

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu dosya eksiksiz [çift dilli ana nottaki](#) kuralları, operator karar mekanizmasına dönüştürür. Hedef her expression için şu sırayı otomatik uygulamaktır: **precedence** → **operand type** → **promotion** → **evaluation** → **assignment**. Dil çalışması: [vocabulary](#).

Kapsam denetimi

Ana çift dilli kaynak promotion, casting, compound assignment, precedence ve side effect örneklerini tam kapsar. Bu hafıza notu şu ayrımları tek çerçevede birleştirir:

- unary ve binary numeric promotion farkı,
- constant narrowing ile runtime narrowing ayrımı,
- `&&/||` ile `&/|` evaluation farkı,
- overflow'un exception üretmemesi,
- equality ve ternary expression'larda compile-time type kontrolü.

1. Operator çözme algoritması

Bir expression gördüğünde sonucu tahmin etmeye başlamadan önce:

- 1 **Parantezle:** Precedence ve associativity'yi belirle.
- 2 **Type yaz:** Her operand'ın compile-time type'ini kenara yaz.
- 3 **Promote et:** Unary/binary numeric promotion uygula.
- 4 **Evaluate et:** Left-to-right operand evaluation ve short-circuit'i izle.
- 5 **Geri ata:** Sonucun hedef variable'a implicit sığıp sığmadığını kontrol et.

Hafıza cümlesi: Önce işlem değil, önce type.

2. Sınavlık precedence merdiveni

Yukarıdaki daha önce uygulanır:

```
postfix      x++ x--
unary        ++x --x + - ! ~ cast
multiplicative * / %
additive     + -
shift        << >> >>>
relational   < > <= >= instanceof
equality     == !=
bitwise/logical & ^ |
short-circuit && ||
ternary      ?:
assignment   = += -= *= ...
arrow        ->
```

Parentheses type kurallarını değiştirmez; yalnız grouping/evaluation order'ı değiştirir.

3. Numeric promotion: en geniş kazanır

Binary numeric expression için:

- 1 Operand'lardan biri `double` ise ikisi `double`,
- 2 değilse biri `float` ise ikisi `float`,
- 3 değilse biri `long` ise ikisi `long`,
- 4 aksi hâlde ikisi `int` olur.

Bu nedenle `byte`, `short` ve `char` çoğu arithmetic işlemde `int`e yükselir.

```
short a = 3;
short b = 4;
// short c = a + b; // Does not compile: result int
int c = a + b;      // 7
```

Unary `+`, unary `-` ve `~`, küçük integral type'ları `int`e promote eder. `++` ve `--` ise assignment içerdiğinden sonucu original variable type'ında tutar.

4. String concatenation: soldan sağa type izi

Binary `+`, iki numeric operand ile addition; operand'lardan en az biri `String` olduğunda concatenation yapar. Aynı precedence seviyesindeki `+` operator'ları left-associative olduğundan ara result type'ını soldan sağa izle:

```
System.out.println(1 + 2 + "c"); // 3c
System.out.println("c" + 1 + 2); // c12
System.out.println("c" + (1 + 2)); // c3
```

Hafıza cümlesi: Soldan giderken `String`e değdiğin anda devamı concatenation olur; parentheses yeni bir ada kurar.

5. Assignment, constant narrowing ve compound assignment

Compile-time constant representable ise küçük type'a explicit cast olmadan atanabilir:

```
byte a = 100;          // constant value byte aralığında
final int n = 10;
byte b = n;            // n constant variable
int runtime = 10;
// byte c = runtime; // Does not compile
```

Compound assignment implicit cast içerir:

```
short s = 3;
s += 7;                // yaklaşık: s = (short) (s + 7)
s++;                  // short olarak kalır
```

`s = s + 7;` ise right side `int` olduğu için derlenmez.

Hafıza cümlesi: += daraltmayı saklar; + saklamaz.

Assignment expression, yeni atanan value'yu üretir:

```
boolean bear = false;
boolean polar = (bear = true); // bear=true, polar=true
```

Bu nedenle = ile equality operator == görünüş olarak benzer olsa da davranışları tamamen farklıdır.

6. Prefix, postfix ve side effect

```
int x = 5;
int a = x++; // a=5, sonra x=6
int b = ++x; // önce x=7, sonra b=7
```

Java operand'ları left-to-right evaluate eder. Precedence grouping'i belirler; operand evaluation sırasını tersine çevirmez.

7. Short-circuit ve boolean operators

- **&& · logical AND:** Sağ operand, sol taraf **false** ise atlanır.
- **|| · logical OR:** Sağ operand, sol taraf **true** ise atlanır.
- **& · logical AND:** İki operand da her zaman değerlendirilir.
- **| · logical OR:** İki operand da her zaman değerlendirilir.
- **^ · XOR:** İki operand da her zaman değerlendirilir; değerler farklıysa sonuç **true** olur.

```
int x = 0;
boolean a = false && ++x > 0; // x hâlâ 0
boolean b = false & ++x > 0; // x artık 1
```

&, |, ^ integral operand'larla bitwise; boolean operand'larla logical çalışır. ! yalnız boolean, ~ yalnız integral operand kabul eder.

8. Equality ve relational kontrolü

- Numeric == binary numeric promotion uygular.
- Boolean yalnız boolean ile equality karşılaştırmasına girer.
- Reference == object identity karşılaştırır; iki type cast-convertible olmalıdır.
- **null** bir reference ile ==/!= kullanılabilir.
- <, >, <=, >= boolean veya arbitrary object kabul etmez.
- Content karşılaştırması genellikle **.equals()** ile yapılır.

9. Ternary operator

```
int score = 80;
String result = score >= 50 ? "pass" : "fail";
```

Condition boolean olmalıdır. Yalnız seçilen branch runtime'da evaluate edilir; fakat iki branch'in type compatibility kontrolü compile time'da yapılır. Ternary right-associative'dir:

```
a ? b : c ? d : e → a ? b : (c ? d : e)
```

10. Overflow ve division traps

Integral overflow exception üretmez; value iki'nin tümleyeni aralığında sarar.

```
int max = Integer.MAX_VALUE;
System.out.println(max + 1); // -2147483648
```

- Integer division fractional kısmı truncate eder: $7 / 2 == 3$.
- Integer / 0 → `ArithmeticException`.
- Floating-point / 0.0 → `Infinity`, `-Infinity` veya `NaN`.
- Remainder'ın işareti left operand ile ilişkilidir: $-7 \% 3 == -1$.

OCP için 20 saniyelik analiz sırası

- 1 Expression'ı precedence'a göre grupla.
- 2 Prefix/postfix değişikliklerini ayrı not et.
- 3 Numeric result type'ını belirle.
- 4 Short-circuit yüzünden atlanan branch var mı?
- 5 Assignment'ta narrowing/cast gerekiyor mu?
- 6 Compile başarılıysa overflow veya runtime exception var mı?

Aktif hatırlama · Özgün çalışma soruları

- 1 `byte b=1; b=b+1;` neden derlenmez, `b+=1;` neden derlenir?
- 2 `false && sideEffect()` çağrısında method çalışır mı?
- 3 `short + short` expression type'ı nedir?
- 4 `Integer.MAX_VALUE + 1` exception üretir mi?

Cevaplar

- 1 `b+1` `int` tir; compound assignment implicit narrowing cast uygular.
- 2 Hayır; `&&` short-circuit yapar.
- 3 `int`.
- 4 Hayır; integral overflow wraparound üretir.